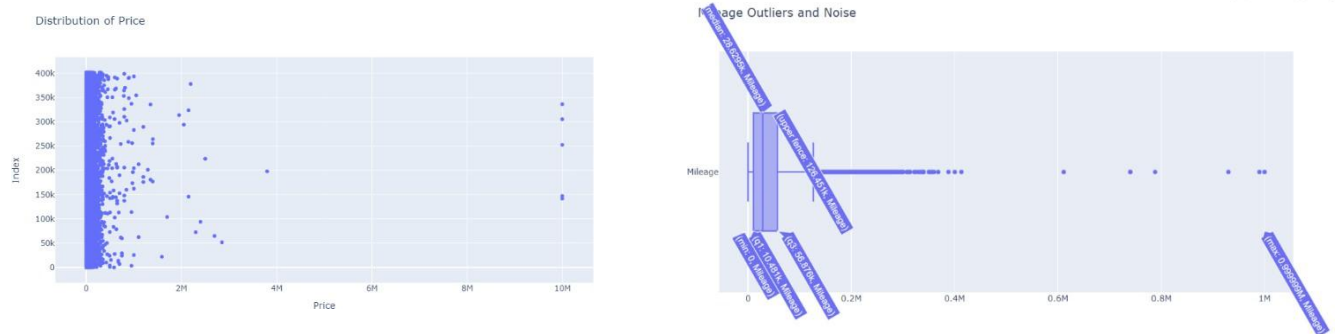


1. Data Processing for Machine Learning

Cars is a dataset containing information about 402005 cars, including details such as the car's mileage, registration code, color, make and model, condition, year of registration, price, body type, and fuel type. The dataset contains 12 columns in total, with each column representing a specific attribute or feature of the cars. Utilising different data exploration techniques, such as the `info()`, `shape()`, and `isnull()` functions to understand the dataset. Plotly graphs are used to explore outliers and noise to find any unique or unexpected patterns in the data.



There are a lot of outliers in the 'price' column, with 26269 out of 402005 values deviating from the normal range. There may be a lot of fluctuation in the costs of the cars in the dataset, as the minimum outlier value, which is 38761, is much greater than the minimum non-outlier value.

1.2 Filling null values :

Using pipeline to fill all the null values by `SimpleImputer` function from the `scikit-learn` library, with the `strategy` argument set to `"most_frequent"`. Using a `for` loop to iterate each given column receives the pipeline applied to it using the `fit_transform` method. Another way to fill null values with specific condition for example filling null values with equal probabilities by taking the top values of respective features such as Colour, Body type, Fuel type.

```
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent'))
])

columns_to_impute = ['standard_colour', 'body_type', 'fuel_type']

for col in columns_to_impute:
    cars[col] = pipeline.fit_transform(cars[[col]])

: colors = ['Black', 'White', 'Grey', 'Blue', 'Silver']
cars['standard_colour'].fillna(value=pd.Series(np.random.choice(colors, size=len(cars), p=[1/5,1/5,1/5,1/5,1/5])),inplace=True)
```

1.3 Dealing with noise/outliers :

On inspection we came to know there are only 127 null values in Mileage column out of 402005 rows. And on further investigation all these 127 cars are marked as USED in vehicle_condition. As it irrelevant to have null values in mileage for a used car so dropping these 127 rows from the dataset.

```
cars = cars.dropna(subset=["mileage"])
```

As we know when a car condition is NEW, mileage should not be greater than 0. So, replacing all the data with correct values by using the below code:.

```
condition = (cars['vehicle_condition'] == 'NEW') & (cars['mileage'] > 0)
cars.loc[condition, 'mileage'] = 0
```

To fill the year_of_registration column using the reg_code, we can extract the relevant year information from the reg_code. From the below article creating a dictionary of keys as reg_code and value as year_of_registration.

https://en.wikipedia.org/wiki/Vehicle_registration_plates_of_the_United_Kingdom

```
cars['year_of_registration']=cars['year_of_registration'].fillna(-1)
cars['year_of_registration']=cars['year_of_registration'].astype('int64')
```

```
dm={'A':1983,'B':1984,'C':1985,'CA':1985,'D':1986,'E':1987,'F':1988,'G':1989,'H':1990,'J':1991,
    'L':1993,'M':1994,'N':1995,'P':1996,'R':1997,'S':1998,'T':1999,'V':2000,'Y':2001} #Dictionary
```

```
for i in range(len(cars)):
    if (cars.loc[i, "year_of_registration"]!=-1 and pd.isna(cars.loc[i, "reg_code"]) and cars.loc[i, "mileage"]==0:
        cars.loc[i, 'year_of_registration']=2020
        cars.loc[i, 'reg_code']='NEW'
    if (cars.loc[i, "year_of_registration"]!=-1 and not pd.isna(cars.loc[i, "reg_code"])):
        if cars.loc[i, 'reg_code'] in dm:
            cars.loc[i, 'year_of_registration']=dm[cars.loc[i, 'reg_code']] #Noise
    if (cars.loc[i, "year_of_registration"]!=-1 and (cars.loc[i, "reg_code"]).isalnum()):
        cars.loc[i, 'year_of_registration']=cars['year_of_registration'].median()
```

Now running a for loop length of cars dataset

- Assigning the year registration with respect to reg code in dictionary
- Filling the null values in reg code as NEW
- If reg code is null and year of registration is null then filling it with median

1.4 Encoding categorical features:

```
: cars_v1 = cars.copy()
```

Copying every section changes into a new dataset with appropriate name. So that it can be used as a backup in future. Using target encoding to replace each categorical feature with average value of target variable's value.

```
: # pip install category_encoders (Just in case needed)

# Target encoding

import category_encoders as ce

# define the target encoding and fit on the data
target_encoder = ce.TargetEncoder(cols=['standard_colour', 'standard_make', 'standard_model', 'vehicle_condition', 'body_type',
encoded_df = target_encoder.fit_transform(cars_v1, cars_v1['price'])
```

1.5 Rescaling & Splitting the data:

Filtering the data to only include the top 15 car brands by frequency in the standard_make column, sample a subset of the data, and split it into training and testing sets for use in a machine learning model. The first function in the pipeline filters the data to only include rows where the standard_make column is in the top 15 brands. The second function in the pipeline samples the data and third line split the data.

```
: # Sampling the dataset and splitting into predictors & target using pipeline

from sklearn.preprocessing import FunctionTransformer

top_brands = cars['standard_make'].value_counts().nlargest(15).index

pipeline = Pipeline([
    ('brand_filter', FunctionTransformer(lambda x: x[x['standard_make'].isin(top_brands)])),
    ('sample', FunctionTransformer(lambda x: x.sample(frac=0.1, replace=True))),
    ('split', FunctionTransformer(lambda x: (x.drop(columns=['price']), x['price'])))
])

X, y = pipeline.fit_transform(cars_v1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

2. Feature Engineering

The below code adds two new features, age and mileage_per_year. The first feature age is calculated by subtracting the year_of_registration of each car to 2022. The second feature mileage_per_year is derived by dividing the mileage of each car by its age and rounding it off up to 2 decimal floating point.

```
: cars_v2['age'] = 2022 - cars_v2['year_of_registration'] # age of the car
cars_v2['mileage_per_year'] = cars_v2['mileage'] / cars_v2['age'] # calculating mileage per year
cars_v2['mileage_per_year'] = cars_v2['mileage_per_year'].round(2) # rounding off the floating value
```

2.1 Polynomial Features:

In this task, two pipelines are created to predict the target variable of a dataset using different feature engineering techniques. The first pipeline is created without polynomial features, while the second pipeline includes polynomial features. After creating both pipelines, the test set is used to predict the target variable using each pipeline, and the RMSE Root Mean Square Error is calculated for both cases. The RMSE is a measure of the average difference between the predicted values and the actual values in the test set.

The output shows that the RMSE without polynomial features is 5778.8692, while the RMSE with polynomial features is 5045.27223. This means that the second pipeline with polynomial features has a lower RMSE, indicating that it performs better in predicting the target variable compared to the first pipeline without polynomial features. This result suggests that adding polynomial features has improved the predictive power of the model by capturing non-linear relationships between the features and the target variable.

```
# Define the pipeline without polynomial features
pipe_no_poly = Pipeline(steps=[
    ('est', LinearRegression())
])

# Fit the pipeline without polynomial features on the training data
pipe_no_poly.fit(X_train, y_train)

# Predict on the test set without polynomial features
y_pred_no_poly = pipe_no_poly.predict(X_test)

# Calculate RMSE without polynomial features
rmse_no_poly = mean_squared_error(y_test, y_pred_no_poly, squared=False)
print("RMSE without polynomial features:", rmse_no_poly)

# Define the pipeline with polynomial features
pipe = Pipeline(steps=[
    ('poly', PolynomialFeatures(degree=2, include_bias=False)),
    ('est', LinearRegression())
])

# Fit the pipeline with polynomial features on the training data
pipe.fit(X_train, y_train)

# Predict on the test set with polynomial features
y_pred_poly = pipe.predict(X_test)

# Calculate RMSE with polynomial features
rmse_poly = mean_squared_error(y_test, y_pred_poly, squared=False)
print("RMSE with polynomial features:", rmse_poly)

RMSE without polynomial features: 5778.869274545653
RMSE with polynomial features: 5045.272238569016
```

2.3 Polynomial Features with interactions:

The output shows that the RMSE without interaction features is 5045.27223, while the RMSE with interaction features is 5085.37. This means that the first pipeline without interaction features performs better in predicting the target variable compared to the second pipeline with interaction features. This result suggests that the interaction features added in the second pipeline may have introduced noise or overfitting, leading to a reduction in performance on the test set.

```
: # create a pipeline without interactions
pipe_no_interaction = Pipeline(
    steps=[
        ('poly', PolynomialFeatures(include_bias=False)),
        ('est', LinearRegression())
    ]
)

# fit the pipeline on the training data
pipe_no_interaction.fit(X_train, y_train)

# predict the target values for the test data
y_pred_no_interaction = pipe_no_interaction.predict(X_test)

# calculate the RMSE score for the predictions without interactions
rmse_no_interaction = mean_squared_error(y_test, y_pred_no_interaction, squared=False)

print('RMSE without interactions:', rmse_no_interaction)

# Pipeline with polynomial features and Ridge regression
interaction = Pipeline(steps=[
    ('poly', PolynomialFeatures(interaction_only=True, include_bias=False))
]).set_output(transform="pandas")

pipe_interaction = Pipeline(
    steps=[
        ('pp', interaction),
        ('est', LinearRegression())
    ]
)

# Fit the pipeline to the training data
pipe_interaction.fit(X_train, y_train)

# Make predictions on the testing data
y_pred = pipe_interaction.predict(X_test)

# Calculate RMSE
rmse = mean_squared_error(y_test, y_pred, squared=False)
print(f"RMSE with polynomial features and interactions: {rmse:.2f}")

RMSE without interactions: 5045.272238569016
RMSE with polynomial features and interactions: 5085.37
```

3. Feature Selection

Manually selected few features from the domain knowledge and build a Ridge regressor model. Making predictions on test data and evaluating rmse score. RMSE score for manually selected features is 5525.9161

Manual Selection

```
X1 = X[['mileage', 'standard_make', 'standard_colour', 'standard_model']]

X_train_man, X_test_man, y_train_man, y_test_man = train_test_split(X1,
                                                                    y,
                                                                    test_size=0.2,
                                                                    random_state=42)

def calculate_rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

model = Ridge().fit(X_train_man, y_train_man)
calculate_rmse(y_test_man, model.predict(X_test_man))

5525.9161798909245
```

3.1 Automated Feature Selection (AFS):

SelectKBest is a method used for selecting the k best features in a dataset. As shown in the picture, these features are considered to be the most important features for predicting the target variable based on the SelectKBest

```
selector = make_pipeline(
    SelectKBest(f_regression, k=7)
).set_output(transform='pandas').fit(X, y)
X_sel = selector.transform(X)

X_sel.columns

Index(['mileage', 'standard_make', 'standard_model', 'year_of_registration',
       'body_type', 'age', 'mileage_per_year'],
      dtype='object')
```

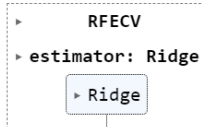
3.2 Recursive Feature Elimination (RFE):

It is a method that selects features by recursively considering smaller and smaller sets of features. Below are the features which are selected by using RFE. The plot is used to determine the optimal number of features to select for the model. In this case, the plot shows that selecting 11 features gives the highest mean test score for cars dataset.

```
from sklearn.feature_selection import RFECV
```

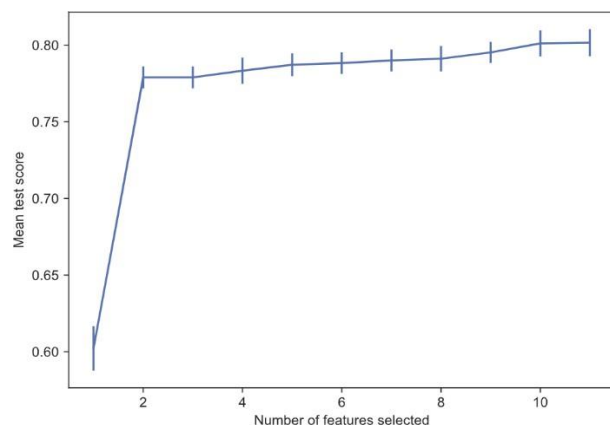
```
model = Ridge()
ref_selector = RFECV(model, step=1, cv=5)
```

```
ref_selector.fit(X, y)
```



```
ref_selector.get_feature_names_out()
```

```
array(['mileage', 'standard_colour', 'standard_make', 'standard_model',
       'vehicle_condition', 'year_of_registration', 'body_type',
       'crossover_car_and_van', 'fuel_type', 'age', 'mileage_per_year'],
      dtype=object)
```



3.3 Sequential Feature Selection (SFS):

```
sfs_forward = SequentialFeatureSelector(  
    Ridge(), n_features_to_select=7, direction="forward"  
) .fit(X, y)
```

```
sfs_forward.get_feature_names_out()
```

```
array(['mileage', 'standard_make', 'standard_model', 'vehicle_condition',  
      'year_of_registration', 'fuel_type', 'mileage_per_year'],  
      dtype=object)
```

3.4 Dimensionality Reduction PCA:

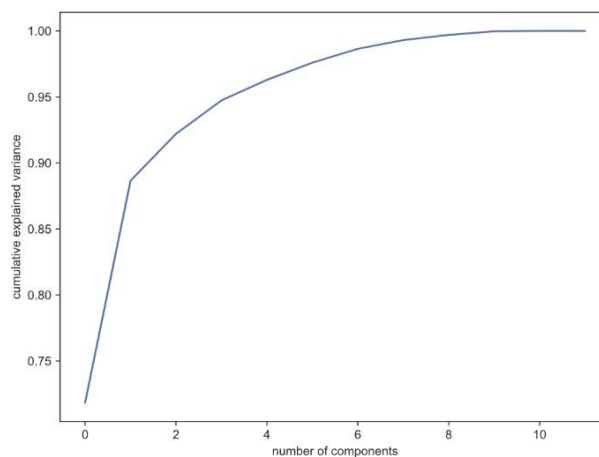
PCA is being performed on the dataset 'cars_v3' without specifying the number of components. This is being done to get a sense of how much variance is explained by each principal component. The 'explained_variance_ratio_' method is used to get the explained variance of each principal component, which is then plotted cumulatively.

```
pca_full = PCA()  
pca_full.fit(cars_v3)
```

▼ PCA
PCA()

```
pca = PCA(n_components=2)  
pca.fit(cars_v3)  
X_pca = pca.transform(cars_v3)  
print("Original shape: {}".format(str(cars_v3.shape)))  
print("Reduced shape: {}".format(str(X_pca.shape)))
```

Original shape: (56765, 12)
Reduced shape: (56765, 2)



Next, PCA is performed again on 'cars_v3', but this time only keeping the first two principal components using 'n_components=2'. The transformed data is stored in 'X_pca'. The code then calculates the feature weights for the first and second principal component using 'pca.components_' and creates a Pandas series with feature names as index and weights as values. The series is then sorted in ascending order of absolute feature weights to identify the most and least important features for the first and second principal components.

Aggregating the features from the above observations:

PCA	mileage, mileage_per_year, standard_model, age, year_of_registration, public_reference, body_type, reg_code
SelectKBest	mileage, mileage_per_year, age, year_of_registration, vehicle_condition
Sequential Feature	mileage_per_year, age, standard_make, standard_model, vehicle_condition, body_type, fuel_type
Recursive Feature	mileage, reg_code, standard_colour, standard_make, standard_model, vehicle_condition, year_of_registration, body_type, crossover_car_and_van, fuel_type, age, mileage_per_year, public_reference

Further selected features for model building : mileage, mileage_per_year, standard_make, standard_model, year_of_registration, age, body_type, fuel_type, standard_colour

4. Model Building

4.1 Linear Model

Building a linear regression model on a dataset and making predictions on the test set. After that, the projected values' Root Mean Squared Error (RMSE) is determined.

```
: from sklearn.linear_model import LinearRegression
  from sklearn.metrics import mean_squared_error

  lr = LinearRegression()
  lr.fit(X_train, y_train)
  y_pred = lr.predict(X_test)
  rmse_linear = mean_squared_error(y_test, y_pred, squared=False)

  print("RMSE with Linear Regressor:", rmse_linear)
```

RMSE with Linear Regressor: 5636.442775943701

From the observations in section 2.2 we came to know that model performs better with polynomial features. So building every model again with polynomial features and checking their RMSE scores

```
poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
```

```
lr_poly = LinearRegression()
lr_poly.fit(X_train_poly, y_train)
y_pred_poly = lr_poly.predict(X_test_poly)
rmse_linear_poly = mean_squared_error(y_test, y_pred_poly, squared=False)

print("RMSE for Linear Regressor with Polynomial Features:", rmse_linear_poly)
```

RMSE for Linear Regressor with Polynomial Features: 5151.626217582397

"RMSE with Linear Regressor: 5636.44", represents the RMSE calculated for the first linear regression model built using the original features. The second output value, "RMSE for Linear Regressor with Polynomial Features: 5151.62", represents the RMSE calculated for the new

linear regression model built using polynomial features. Since the RMSE for the model built with polynomial features is lower than the RMSE for the original linear regression model, we can conclude that the model with polynomial features is better at making predictions.

Ridge & Lasso Regressor

```
: from sklearn.linear_model import Ridge, Lasso

# Ridge regression
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)
rmse_ridge = mean_squared_error(y_test, y_pred_ridge, squared=False)
print("RMSE for Ridge regression:", rmse_ridge)

# Lasso regression
lasso = Lasso(alpha=1.0)
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)
rmse_lasso = mean_squared_error(y_test, y_pred_lasso, squared=False)
print("RMSE for Lasso regression:", rmse_lasso)

RMSE for Ridge regression: 5842.745854635146
RMSE for Lasso regression: 5842.745858837638
```

4.2 Random Forest

```
: from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(random_state=42)
rf_model.fit(X_train, y_train)

rf_rmse = mean_squared_error(y_test, rf_model.predict(X_test), squared=False)
print("RMSE for Random Forest:", rf_rmse)
```

RMSE for Random Forest: 3728.8350614995093

```
poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_poly = poly.fit_transform(X)

X_train_poly, X_test_poly, y_train_poly, y_test_poly = train_test_split(X_poly, y, test_size=0.2, random_state=42)

rf_model_poly = RandomForestRegressor(random_state=42)
rf_model_poly.fit(X_train_poly, y_train_poly)

# Evaluate the Random Forest Regressor model with transformed data
rf_rmse_poly = mean_squared_error(y_test_poly, rf_model_poly.predict(X_test_poly), squared=False)
print("RMSE for Random Forest with polynomial features:", rf_rmse_poly)
```

RMSE for Random Forest with polynomial features: 3816.5293015658403

The RMSE for the random forest regressor without polynomial features was 3728.83, and with polynomial features, it was 3816.52. This indicates that adding polynomial features did not improve the model's performance in this case.

4.3 Boosted Tree:

```
from sklearn.ensemble import GradientBoostingRegressor

bt_model = GradientBoostingRegressor(random_state=42)
bt_model.fit(X_train, y_train)
y_pred = bt_model.predict(X_test)

# Calculate the root-mean-square error of the model
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print('RMSE for GradientBoostingRegressor:', rmse)
```

RMSE for GradientBoostingRegressor: 4414.914131290602

```
from sklearn.experimental import enable_hist_gradient_boosting
from sklearn.ensemble import HistGradientBoostingRegressor

ht_model = HistGradientBoostingRegressor().fit(X_train, y_train)
y_pred = ht_model.predict(X_test)
rmse = mean_squared_error(y_test, y_pred, squared=False)

print("RMSE for HistGradientBoostingRegressor:", rmse)
```

C:\Users\ksk78\anaconda3\lib\site-packages\sklearn\experimental\en
1.0, it is not needed to import enable_hist_gradient_boosting anym
egressor are now stable and can be normally imported from sklearn.
warnings.warn()

RMSE for HistGradientBoostingRegressor: 4020.7607588930623

GradientBoostingRegressor and HistGradientBoostingRegressor are two regression models that use boosting methods. The HistGradientBoostingRegressor model has a lower RMSE (Root Mean Squared Error) than the GradientBoostingRegressor model, indicating that the former performs better in terms of prediction accuracy. This might be as a result of HistGradientBoostingRegressor's use of histogram-based gradients, which are more effective and reliable than GradientBoostingRegressor's use of conventional gradient-based approaches.

```
: pipe_poly = Pipeline([
    ('poly', PolynomialFeatures(interaction_only=True, include_bias=False)),
    ('model', GradientBoostingRegressor(random_state=42))
])

# Fit the model and calculate the RMSE score with polynomial features
pipe_poly.fit(X_train, y_train)
rmse_poly = mean_squared_error(y_test, pipe_poly.predict(X_test), squared=False)
print("RMSE for GradientBoostingRegressor with polynomial features: ", rmse_poly)
```

RMSE for GradientBoostingRegressor with polynomial features: 4348.966524429667

```
: pipe = Pipeline([
    ("poly", PolynomialFeatures(degree=2, include_bias=False)),
    ("estimator", HistGradientBoostingRegressor(random_state=42))
]).set_output(transform="pandas")

# transform the data with polynomial features
X_train_poly = pipe["poly"].fit_transform(X_train)
X_test_poly = pipe["poly"].transform(X_test)

# fit the model with transformed data
pipe.fit(X_train_poly, y_train)

# calculate the RMSE with transformed data
rmse_poly = mean_squared_error(y_test, pipe.predict(X_test_poly), squared=False)
print("RMSE for HistGradientBoostingRegressor with polynomial features:", rmse_poly)
```

RMSE for HistGradientBoostingRegressor with polynomial features: 3812.753932162004

Comparing the RMSE values, it can be observed that the use of polynomial features improves the performance of both the GradientBoostingRegressor and HistGradientBoostingRegressor models, as the RMSE values decrease.

4.4 Model Ranking

Below table give information on respective model and their RMSE score

Model	RMSE Score
Linear Regressor without Polynomial features	5842.745855
Linear Regressor with Polynomial Features	5151.626218
Random Forest without polynomial features	3728.835061
Random Forest with polynomial features	3816.529302
RMSE for Ridge regression	5842.745855
RMSE for Lasso regression	5842.745859
GradientBoostingRegressor without Polynomial features	4414.914131
GradientBoostingRegressor with polynomial features	4348.966524
HistGradientBoostingRegressor without Polynomial features	4020.760759
HistGradientBoostingRegressor with polynomial features	3812.753932

Top 3 model selected based upon the above table information: Random Forest, Hist Gradient Boosting and Gradient Boosting. These models have the lowest RMSE scores, indicating they have better performance compared to the other models evaluated.

4.5 Grid Search for top 3 models:

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import PolynomialFeatures

pipeline = Pipeline(steps=[('poly', PolynomialFeatures()),
                           ('rf', RandomForestRegressor())])

param_grid = {'poly__degree': [1, 2],
              'rf__n_estimators': [50, 100],
              'rf__max_depth': [None, 5, 10]}

grid_search = GridSearchCV(pipeline, param_grid=param_grid, cv=5, n_jobs=-1, verbose=1)
grid_search.fit(X_train, y_train)

print("Best parameters: ", grid_search.best_params_)
```

Fitting 5 folds for each of 12 candidates, totalling 60 fits
Best parameters: {'poly__degree': 1, 'rf__max_depth': None, 'rf__n_estimators': 100}

```
# define the pipeline with polynomial features and random forest
rf_pipeline = make_pipeline(PolynomialFeatures(degree=1),
                             RandomForestRegressor(max_depth=10))

rf_pipeline.fit(X_train, y_train)

y_pred = rf_pipeline.predict(X_test)

rmse = mean_squared_error(y_test, y_pred, squared=False)

print("RMSE score:", rmse)
```

RMSE score: 3741.940142294095

```
gb = GradientBoostingRegressor()

param_grid = {
    'n_estimators': [50, 100, 150],
    'max_depth': [2, 3, 4],
    'learning_rate': [0.05, 0.1, 0.2]}

grid_search = GridSearchCV(gb, param_grid=param_grid, cv=5, n_jobs=-1)
grid_search.fit(X_train, y_train)

best_params = grid_search.best_params_

y_pred = grid_search.predict(X_test)

rmse_score = mean_squared_error(y_test, y_pred, squared=False)

# print best parameters, rmse score
print("Best parameters:", best_params)
print("RMSE score:", rmse_score)
```

Best parameters: {'learning_rate': 0.2, 'max_depth': 4, 'n_estimators': 150}
RMSE score: 3931.9692030741135

```
model = HistGradientBoostingRegressor()

param_grid = {'learning_rate': [0.1, 0.05, 0.01],
              'max_iter': [100, 200, 300],
              'max_depth': [None, 5, 10]}

grid = GridSearchCV(estimator=model, param_grid=param_grid, cv=5, n_jobs=-1)
grid_result = grid.fit(X_train, y_train)

print("Best Parameters: ", grid_result.best_params_)

y_pred = grid_result.best_estimator_.predict(X_test)

print("RMSE: ", np.sqrt(mean_squared_error(y_test, y_pred)))
```

Best Parameters: {'learning_rate': 0.1, 'max_depth': None, 'max_iter': 300}
RMSE: 3901.4234810313633

4.6 Ensemble

From the tabular data 4.4 section combining top 3 models such as Random Forest, Gradient Boosting and Hist Gradient Boosting into Ensemble. The ensemble of Random Forest, GradientBoosting, and Hist GradientBoosting models has the lowest RMSE score of 3030.230698, indicating that the ensemble is the best model for this dataset among all the models considered.

```
from sklearn.ensemble import VotingRegressor

# Define the models with their best configurations
model1 = make_pipeline(PolynomialFeatures(degree=2), RandomForestRegressor(max_depth=None, n_estimators=100))
model2 = GradientBoostingRegressor(n_estimators=150, learning_rate=0.2, max_depth=4)
model3 = HistGradientBoostingRegressor(max_iter=300, learning_rate=0.1, max_depth=10)

# Build the ensemble with the best models
ensemble = VotingRegressor([('rf', model1), ('gb', model2), ('hgb', model3)])

# Fit the ensemble on the data
ensemble.fit(X_train, y_train)

# Make predictions on the test set
y_pred = ensemble.predict(X_test)
rmse = mean_squared_error(y_test, y_pred, squared=False)
print('RMSE:', rmse)

RMSE: 3030.230698436482
```

In summary, the Random Forest model with polynomial features and the ensemble model are the two best performing models for this dataset, while the Linear Regressor and Ridge/Lasso models are not effective for this dataset.

5. Model Evaluation and Analysis

5.1 Overall Performance with Cross-Validation

```
# Cross-validation
cv_scores = {}
for model in [model1, model2, model3, ensemble]:
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_squared_error')
    cv_scores[type(model).__name__] = np.sqrt(-scores)

# Calculate mean and standard deviation of cross-validation scores
for name, scores in cv_scores.items():
    print(f"{name} - Mean : {scores.mean():.2f}, Standard Deviation: {scores.std():.2f}")

RandomForestRegressor - Mean : 3442.33, Standard Deviation: 229.43
GradientBoostingRegressor - Mean : 4144.13, Standard Deviation: 128.08
HistGradientBoostingRegressor - Mean : 3641.90, Standard Deviation: 222.33
VotingRegressor - Mean : 3534.91, Standard Deviation: 205.07
```

Based on the given output, it can be concluded that all four models, namely RandomForestRegressor, GradientBoostingRegressor, HistGradientBoostingRegressor, and VotingRegressor, have performed relatively well. The RandomForestRegressor model has the lowest mean score of 3442.33, which indicates that this model is the most accurate in predicting the target variable. However, the standard deviation of this model is also the highest among all the models, indicating that it may not be as consistent in its predictions as the other models.

The GradientBoostingRegressor model has a mean score of 4144.13 and the lowest standard deviation of 128.08, indicating that it is both accurate and consistent in its predictions.

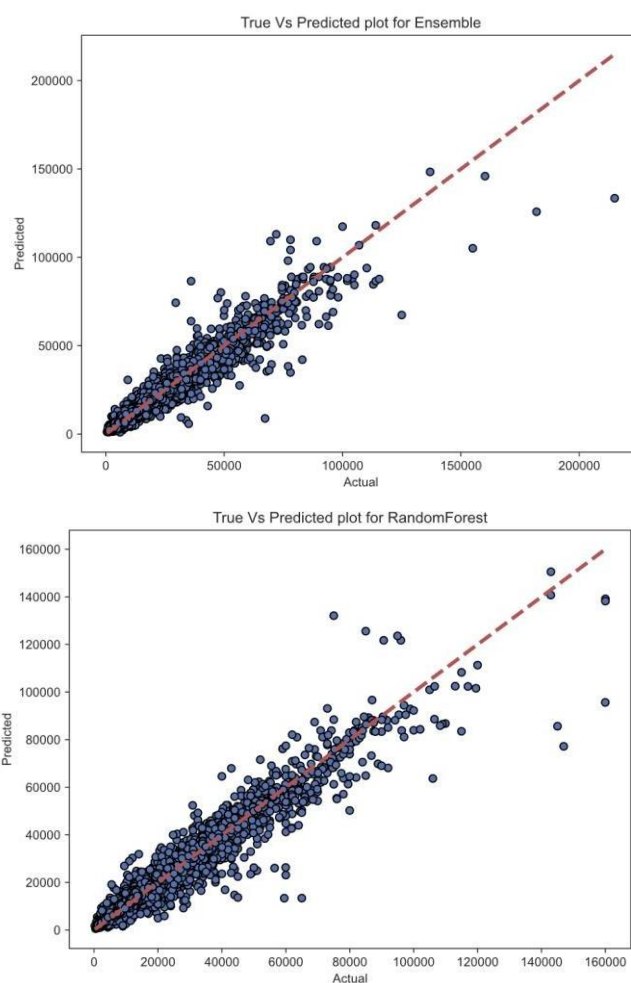
The HistGradientBoostingRegressor model has a mean score of 3641.90 and a standard deviation of 222.33, which is also relatively good.

The VotingRegressor model has a mean score of 3534.91 and a standard deviation of 205.07, which is similar to the HistGradientBoostingRegressor model.

Overall, based on the mean score and standard deviation, it can be concluded that the RandomForestRegressor, GradientBoostingRegressor, and HistGradientBoostingRegressor models are the top three performers and could be considered for further model building. However, the final decision should also consider other factors, such as the computational cost and interpretability of the models.

5.2 True vs Predicted Analysis

The scatter plot is used to visualize the relationship between the actual and predicted values.



The diagonal line represents perfect prediction where the actual and predicted values are the same. A model that predicts perfectly would have all the points on the diagonal line. From the analysis of the scatter plot for both Random Forest and Ensemble models, it appears that both models produce similar results, with minor differences. However, the Ensemble model has more points along the diagonal line, indicating that it has predicted more accurately than the Random Forest model. Although few points for the higher target value are more distributed and not predicted well. Perhaps 80% of the cars data falls under Q1 and Q2 quantile range. As we do not have sufficient information on higher target value, model is unable to predict correctly.

5.3 Global and Local Explanations with SHAP

Creating a sample dataset of 2000 records from the original dataset as SHAP will take longer time to compute values

```
cars_sample = cars_v4.sample(n=2000, random_state=42)

# Split the dataset
X_sample = cars_sample.drop(['price'], axis=1)
y_sample = cars_sample['price']

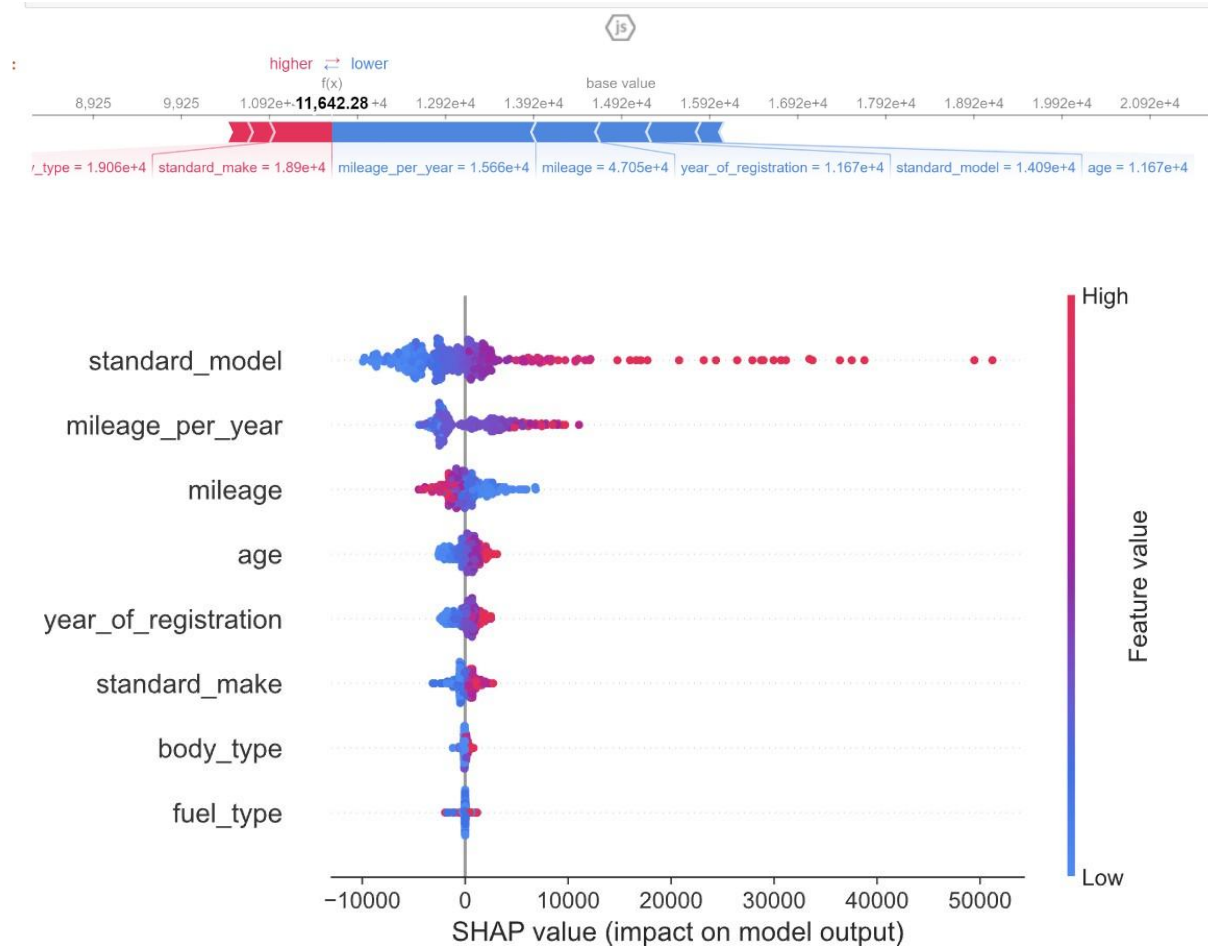
X_train, X_test, y_train, y_test = train_test_split(X_sample, y_sample, test_size=0.2, random_state=42)

import shap #pip install shap (just in cases needed)

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

explainer = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X_test)

shap.initjs()
shap.force_plot(explainer.expected_value, shap_values[0,:], X_test.iloc[0,:])
```



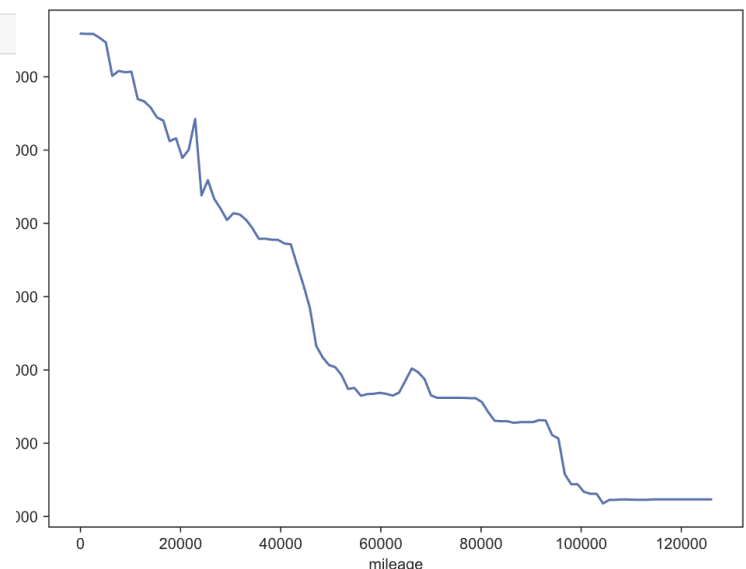
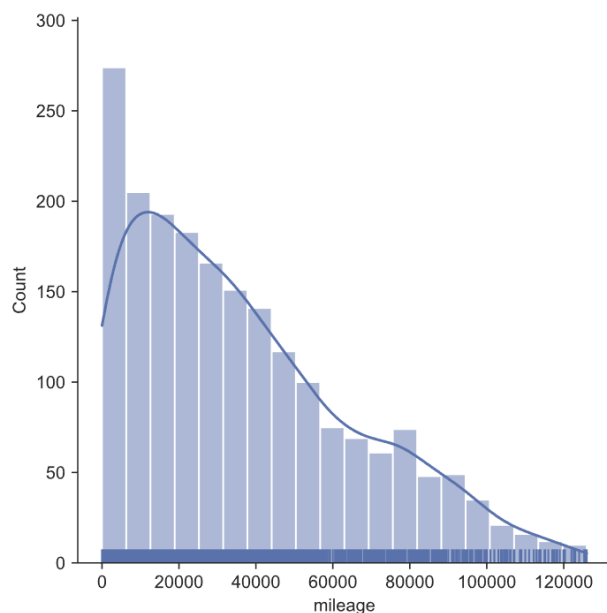
Insights from the summary plot:

- Each dot represents feature from the sample cars dataset
- Red dot represents higher impact on target value. Blue dot represents lower impact
- Position of dot on x-axis represent the intensity of impact it had on target output. The more it moves away the greater the intensity.
- Standard model had greater positive impact on target output and wider range.
- Cumulative blue dots of Standard model represent it had lower impact and it is also clear that red dots are scattered all along on the x axis. There could be a possibility model will overfit if it considers only standard model
- Mileage per year which is a new derived feature shows more positive high impact.
- Mileage feature shows inverse proportion to the target value. It shares equal amount of lower and higher impact. Red dots on negative value represents higher impact which implies the more mileage the less price.
- Age, Year of registration and Standard make shows similar trends. They share equal higher and lower impacts.
- Body type and fuel type are the least features to show impact on the model. They also represent more blue dots, and it may lead to model underfit

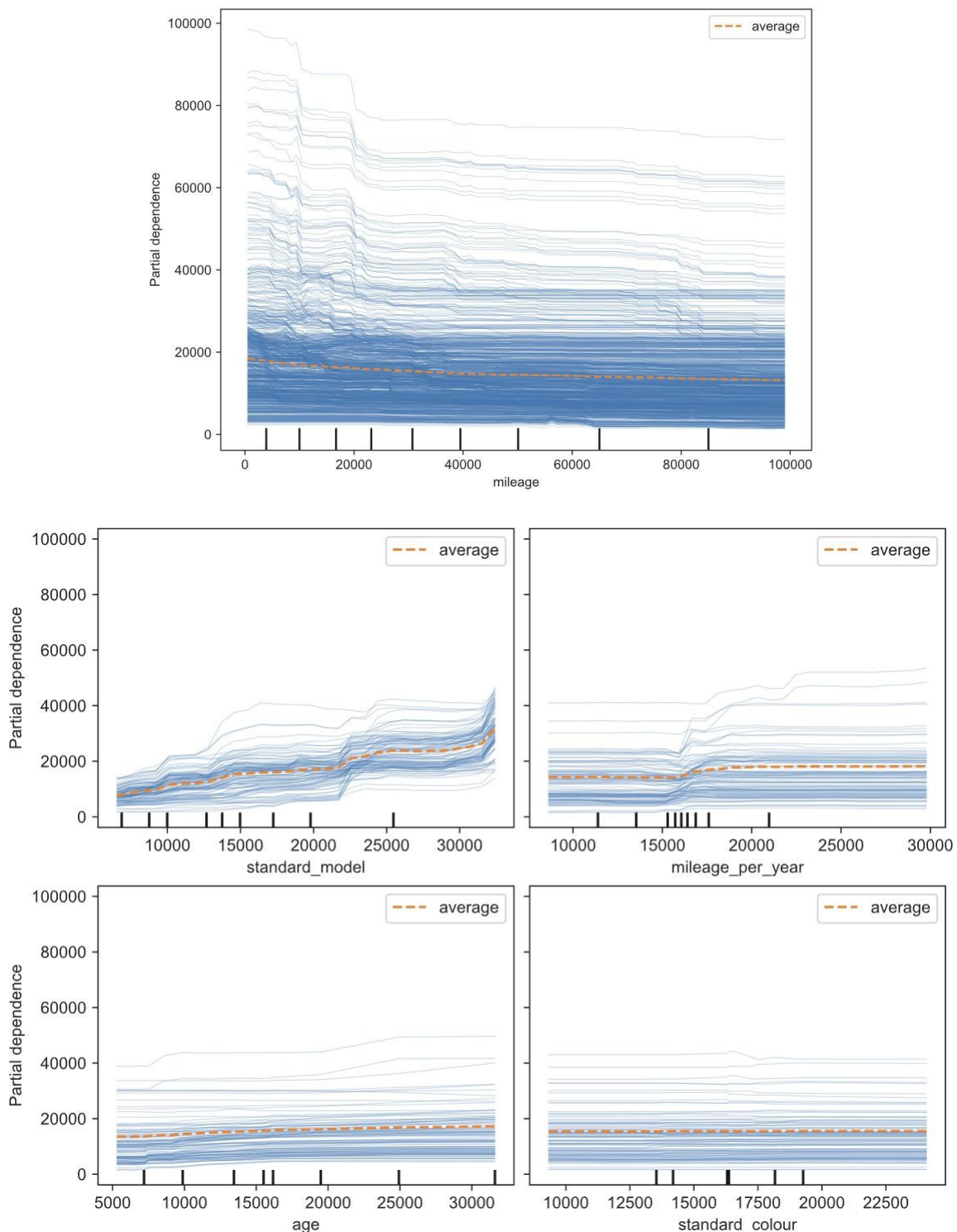
5.4 Individual Conditional Expectation

Generating synthetic data for the mileage feature using the linspace function. This creates a linear space of 100 equally spaced points between the minimum and maximum values of mileage in the original data. After that, a new dataframe is created by merging the synthetic mileage data. And plots are drawn to visualize the distribution of the mileage feature in the synthetic dataset with histogram and line plot

```
sns.displot(X_sample['mileage'], kde=True, rug=True);
```



5.5 Partial Dependency Plot:



The generated plots show the relationship between each feature and the target variable while holding other features constant. Mileage feature shows variations in PDP plots. At beginning it is very high and it is flattened while coming to the average. According to the partial dependence plot, the average relationship between the milage and the target is virtually linear and shows a downward trend. Regards Standard model the relationship fluctuates low and high throughout the graph. Age and Standard colour shows similar pattern and trends.